GRAPH NEURAL NETWORKS FOR FLOOD FORECASTING

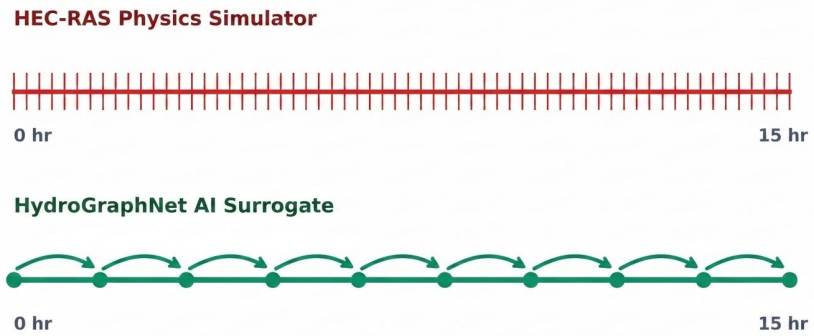
HydroGraphNet

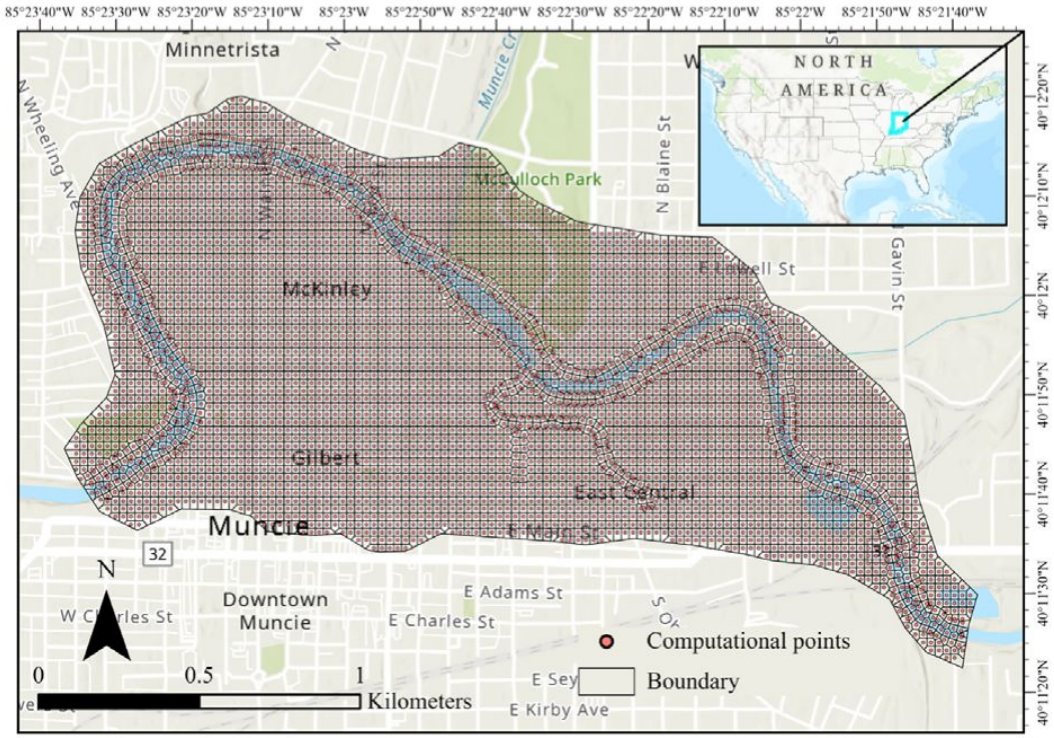
Taghizadeh et al. 2025 · White River, Muncie, Indiana

What Problem Is HydroGraphNet Trying to Solve?

Predicting where floodwater will go — and how deep it will get — hours before a storm peaks

- HEC-RAS solves complex fluid equations at every cell, every time step
- Accurate physics — but takes hours for one flood event
- In an emergency, you need many scenarios in minutes
- The AI learns the same dynamics from past runs, then predicts instantly



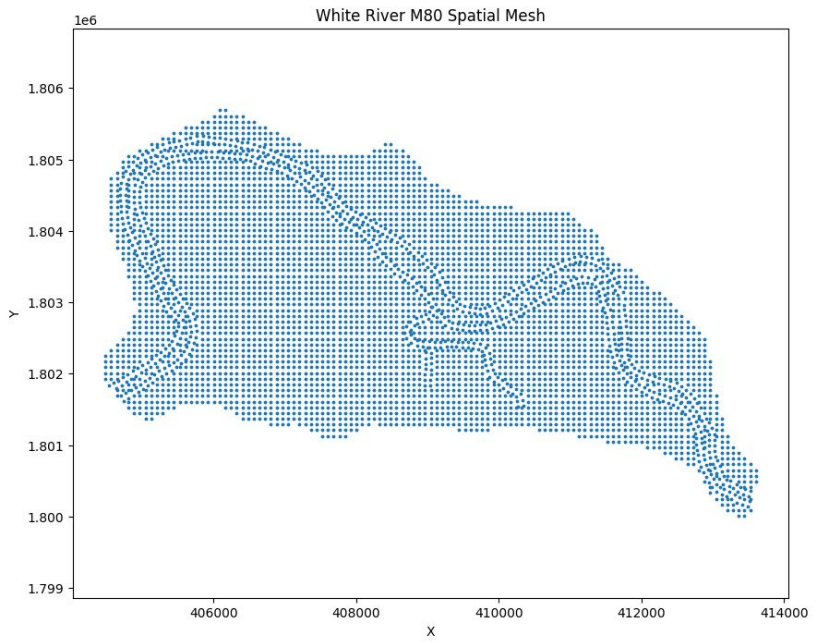


(c) Computation Points

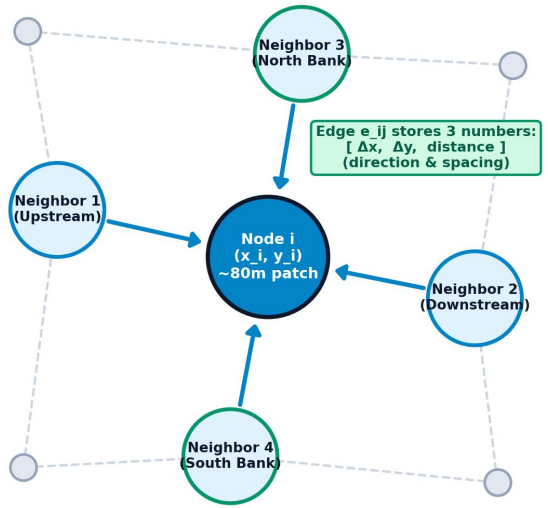
Study Area (Paper Fig. 3c): White River north of Muncie, Indiana — divided into 4,787 small patches where we want to predict water depth.

Turning the Area into a Graph: (X, Y) Mesh -> Nodes & Edges

We treat each patch of land as a Node, and connect nearby patches with Edges so water can flow between them



Zoomed In: Each Dot Is a Node Connected to k = 4 Nearest Neighbors



Why a Graph Instead of a Grid?

- **Rivers Curve & Bend:** A square image grid (CNN) wastes pixels on dry land and blurs narrow river channels.
- **Flexible (X, Y) Points:** Notice in the blue mesh how points follow the winding S-curve of the White River!
- **Matches Real Physics:** Water flows from one patch of land into its immediate neighbors — just like messages along graph edges.

How We Build the Graph G = (Nodes, Edges) Step-by-Step

- **1. Nodes (4,787 Land Patches):** Every blue dot at position (X, Y) is 1 Node (~80 m apart). Each node holds numbers describing its land and its current water.
- **2. Edges (19,148 Neighbor Arrows):** For every node, we find its k = 4 closest neighbors and draw directed arrows between them.
- **3. Edge Features (3 Numbers per Arrow):** Each arrow e_ij stores [Δx, Δy, distance] — telling the model what direction neighbor j is in and how far water must travel!

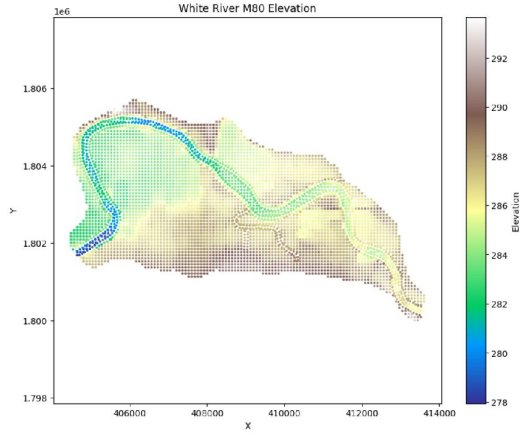
What Does Each Node Know About Its Land? (Height & Shape)

First 6 permanent ('static') features stored inside every node — these describe the terrain and never change over time

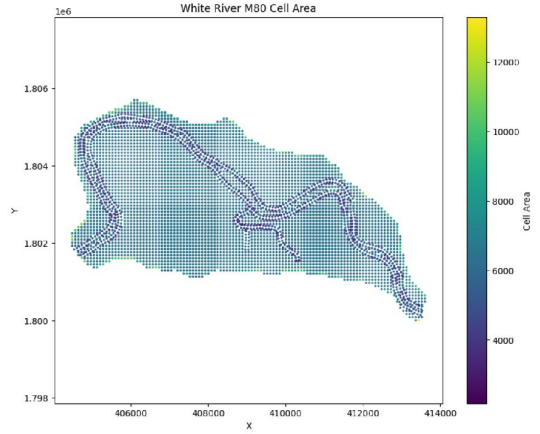
Static Node Features #1 to #6 (Shape of the Ground)

- **#1 & #2 — Coordinates (X, Y):** Exact map position of the node's center (~80 m spacing).
- **#3 — Cell Area (m²):** How big this patch of land is (average ~5,828 m²). A bigger patch needs more cubic meters of water to raise depth by 1 meter!
- **#4 — Elevation (278–294 m):** Height of the ground above sea level. Look at the top-left map: the blue river channel sits lower than the brown hills, so water stays in or near the river first.
- **#5 — Slope (0°–52°):** How steep the ground is. Steep riverbanks make water rush quickly; flat floodplains let water spread out.
- **#6 — Curvature (-30 to +30):** Whether the patch is shaped like a bowl (< 0, blue, where puddles collect) or a ridge (> 0, red, which sheds water).

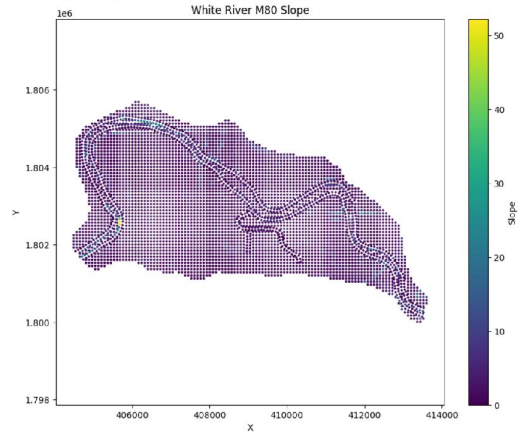
Elevation (m): River channel is low (blue), hills high (brown)



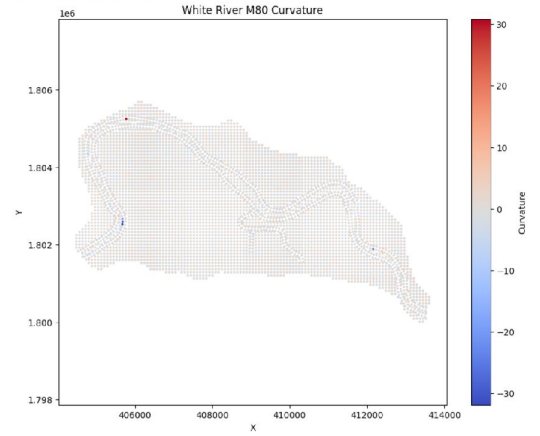
Cell Area (m²): Size of each land patch (~5,800 m² avg)



Slope (deg): Steep riverbanks vs. flat floodplains



Curvature: Concave bowls (<0, blue) collect water



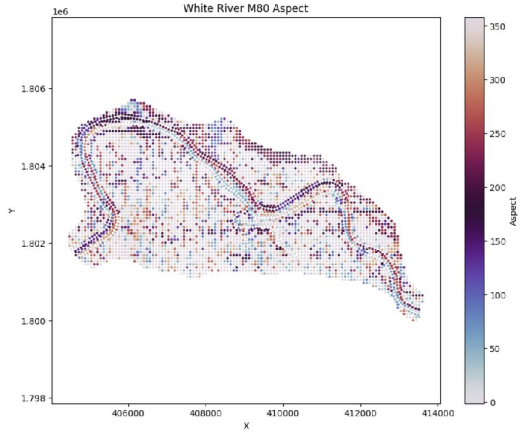
More Land Features: Slope Direction, Friction & Paved Ground

Features #7 to #10 tell each node which way downhill points, how rough the surface is, and whether rain soaks in

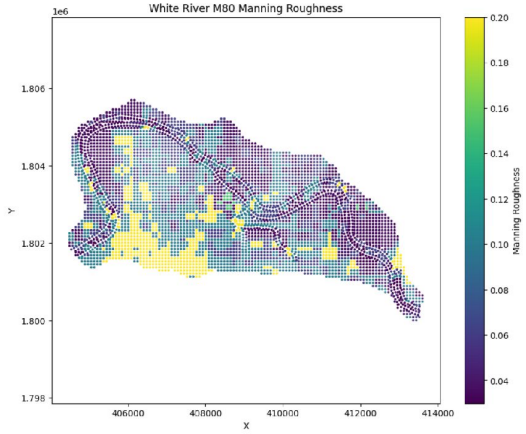
Static Node Features #7 to #10 (Surface & Soil)

- **#7 — Aspect (0°–360°):** Compass direction that a sloped patch faces — tells the model which direction gravity pulls water.
- **#8 — Manning's Roughness n (0.03–0.20):** Surface friction! Smooth riverbeds (dark purple, n ≈ 0.03) let water glide fast; thick grass, trees, and buildings (yellow, n ≈ 0.18) slow floodwater down.
- **#9 — Imperviousness IP (0%–90%):** Percentage of ground covered by pavement, roofs, or roads (bright yellow = urban Muncie). Paved ground cannot absorb rain, creating instant street runoff!
- **#10 — Flow Accumulation:** How many uphill land patches naturally drain through this node along the terrain.

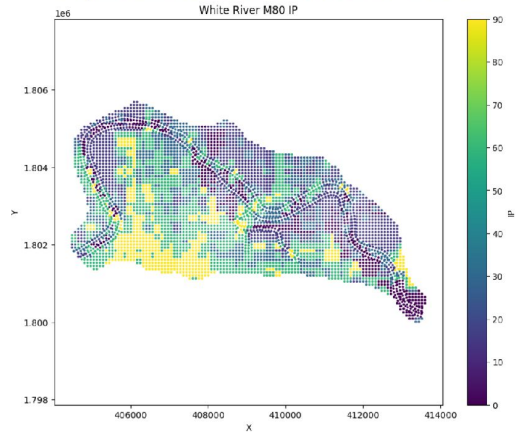
Aspect (0–360°): Downhill compass direction



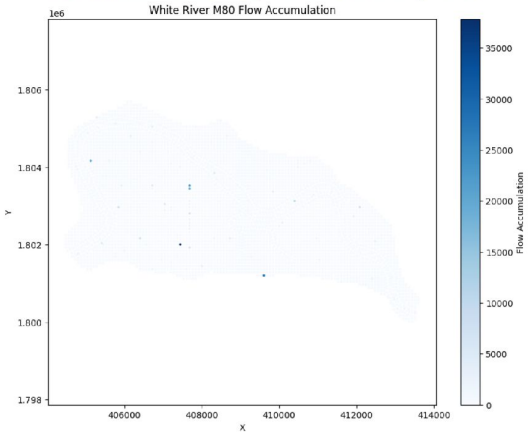
Manning's Roughness n: Smooth river vs. rough land



Imperviousness IP (%): Paved town vs. soil



Flow Accumulation: Uphill cells draining here



Adding Weather & Current Water: All 16 Numbers Inside Each Node

At every 20-minute step t, we combine the 10 land features + 2 storm inputs + 4 recent water measurements

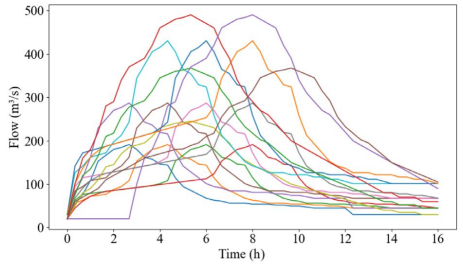
Inside Every Single Node at Time t: A Vector of 16 Numbers

#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	#11	#12	#13	#14	#15	#16
x	y	Area	Elev	Slope	Aspect	Curv	Rough n	FlowAcc	IP %	Inflow	Rain	h_(t-1)	h_t	V_(t-1)	V_t

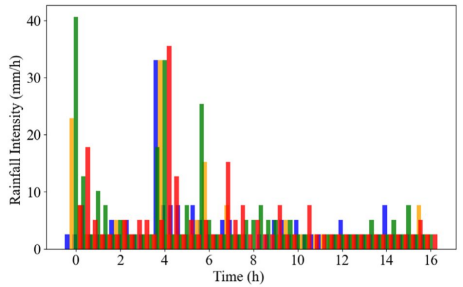
10 Static Terrain Features (Fixed Land Properties) | 2 Weather Inputs | 4 Past Water States (Last 2 Steps)

Why We Add 6 Time-Changing ('Dynamic') Numbers at Step t

- **#11 — Upstream River Inflow (m³/s):** How much water is entering the upstream end of the White River right now (copied to all nodes so the whole map knows a surge is coming).
- **#12 — Rainfall Intensity (mm/h):** How hard rain is falling from the sky over the basin during this 20-minute step.
- **#13 & #14 — Past 2 Water Depths [h_(t-1), h_t]:** How deep the water was 20 mins ago and right now! Giving 2 steps (lookback K = 2) lets the model see whether water is currently RISING or FALLING.
- **#15 & #16 — Past 2 Water Volumes [V_(t-1), V_t]:** Total cubic meters of water sitting in this patch 20 mins ago and right now.



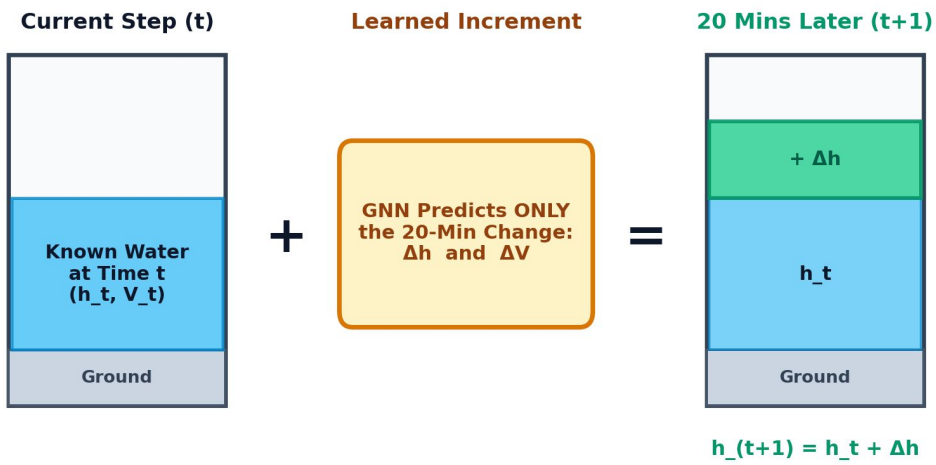
(a) Base Hydrographs



(b) Base Hyetographs

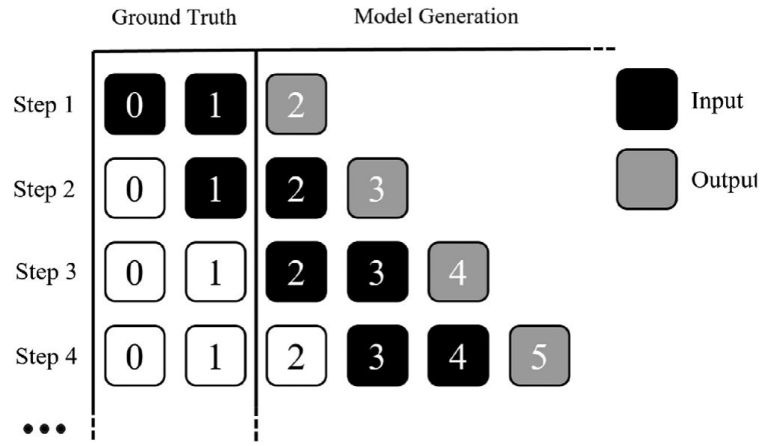
Predicting Water Height at t+1 by Learning the Change (Δh)

Instead of guessing total depth from scratch, the model predicts how much water rises or falls in 20 minutes



1. Single Step: Why Predict Δh Instead of h_(t+1)?

- **Most Nodes Change Slowly:** In 20 minutes, a node with 2.00 m of water might rise to 2.08 m (Δh = +0.08 m), while dry hills change by 0.00 m.
- **Simple Residual Addition:** The GNN outputs two small numbers per node — [Δh, ΔV] — and adds them to the current water:
  h_(t+1) = h_t + Δh and V_(t+1) = V_t + ΔV
- **Much Easier to Learn:** Starting from h_t via a shortcut ('residual connection') keeps predictions anchored to reality.

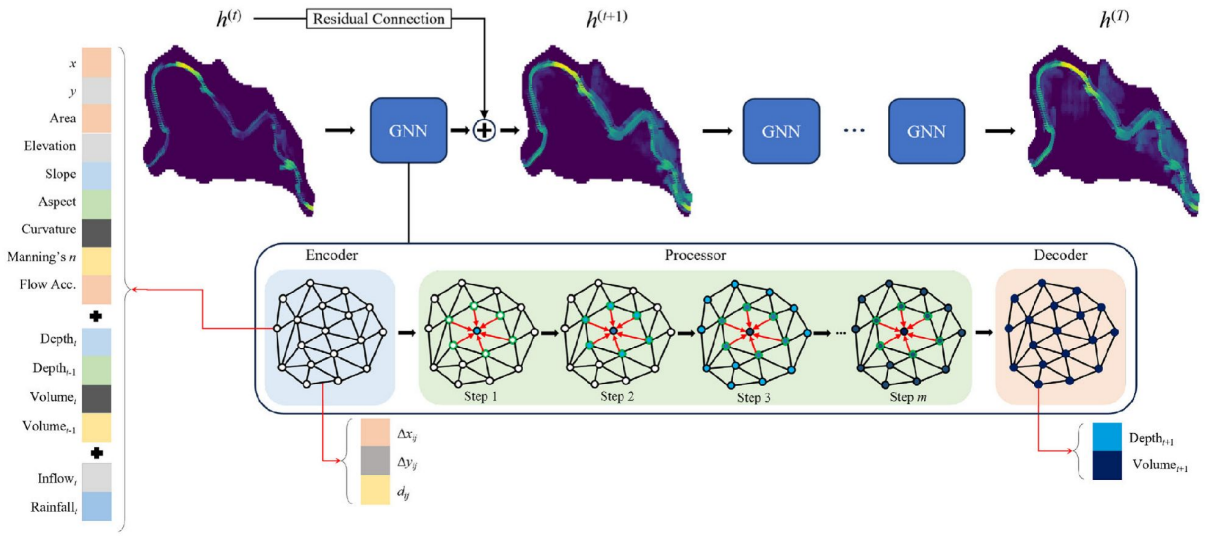


2. Multi-Hour Forecast: Sliding-Window Rollout (Left)

- **Step 1:** Take real starting water at [t=0, t=1] -> predict water at t=2 (grey box).
- **Step 2:** Slide window forward! Use [t=1, predicted t=2] + new rain/inflow -> predict t=3.
- **Repeat 45 to 62 Times:** Each 20-minute jump feeds into the next, forecasting 15 to 20.7 hours of flooding in 0.2 seconds!

How Nodes Talk to Neighbors: Encoder -> Processor -> Decoder

Water moves between neighboring patches in real life — Message Passing mimics that exact neighbor-to-neighbor flow



(b) HydroGraphNet model for flood dynamics forecasting

1. Encoder (Compress Inputs)

- **Node Encoder:** Turns each node's 16 raw features into a 64-dimensional vector.
- **Edge Encoder:** Turns each arrow's 3 distance numbers [Δx, Δy, d] into a 64-dimensional vector.

2. Processor (5 Neighbor Rounds)

- **Pass Water Messages:** Along every arrow, neighbors compare states and send 'water flux' messages, summed up at each node.
- **Why 5 Rounds?** 1 round = hear from immediate neighbors (80 m). 5 rounds = flood waves travel 5 hops (~400 m) in one step!

3. Decoder (Output Δh, ΔV)

- **Read Out the Change:** A small 3-layer MLP reads each node's final 64-dimensional vector and outputs [Δh, ΔV].
- **Add to Current State:** h_(t+1) = h_t + Δh gives the new flood map!

Fourier-KAN

How Taghizadeh et al. (2025) replace the standard MLP node encoder with sine/cosine waves for interpretability

What the Paper's KAN Layer Actually Computes (Paper Eq. 30)

Instead of weights * ReLU, pass each feature x_p through 5 waves:

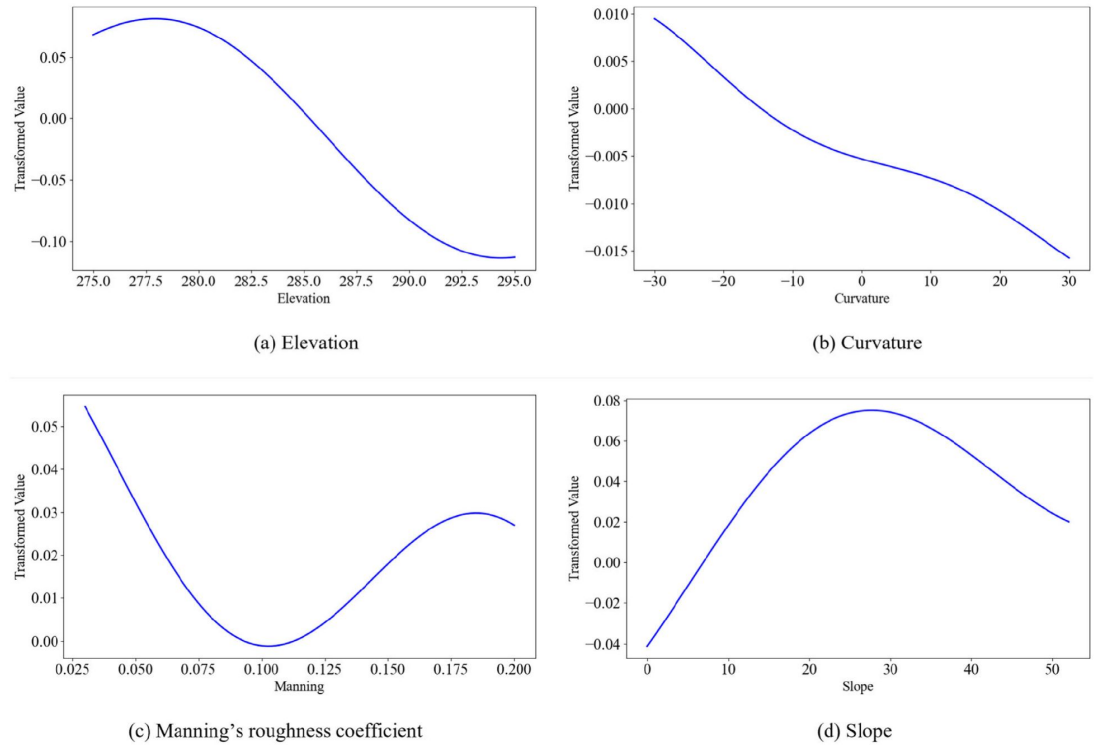
$$\mathbf{KAN}(x_p) = \Sigma^{5}_{k=1} \left[a_{p,k}\cos(kx_p) + b_{p,k}\sin(kx_p) \right]$$

Why the Authors Used KAN

- The main idea is that, in a conventional MLP, the input is transformed through a series of weight matrices and nonlinear activation functions. It is difficult to directly interpret how individual input features are transformed, especially with ReLU, whose piecewise-linear behaviour can make the learned transformations less intuitive to visualise.
- In this case, the model learns explicit functions associated with the input features. We can plot these functions and observe how the model transforms each input as its value changes. This provides a more interpretable view of the learned representations, helping us understand how the model responds to different input features without having to inspect the internal weight matrices.

Paper Table 4: Reported Test Results With vs. Without KAN

Paper Variant	RMSE Error ↓	NSE Score ↑	CSI @ 0.30m ↑
Without KAN (Standard MLP)	0.27 m	0.80	0.77
With KAN (Fourier K=5)	**0.18 m**	**0.89**	**0.87**



Paper Fig. 9: Learned 1D Fourier-KAN curves for 4 terrain features.

How the Paper Trains HydroGraphNet – and Its Reported Results

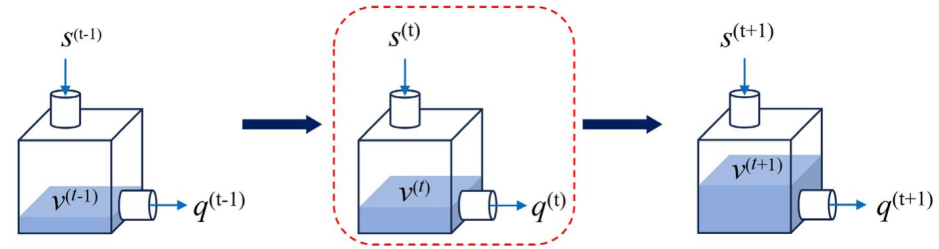
Combining node data loss, two physics penalties, and 2-step 'pushforward' unrolling across 500 HEC-RAS storms

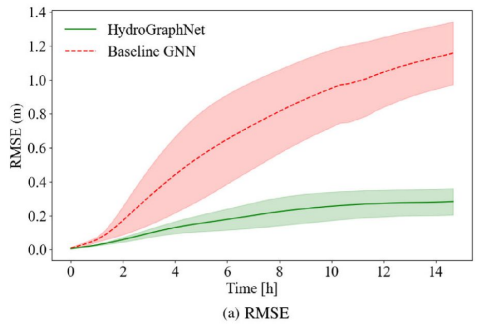
Four Training Ingredients Proposed in the Paper

- **1. Node Prediction Loss (L_{pred}):** Standard Mean Squared Error — make predicted water depth h and volume V match the HEC-RAS simulator at every node.
- **2. Whole-Basin Water-Tank Loss ($L_{physics}$):** Treat the whole map as one giant bucket (top-right): total water added across all 4,787 nodes cannot exceed [River Inflow + Rain - Soil Soaking]. Uses one-sided ReLU(x)² because unmeasured river water also exits downstream.
- **3. Cell Geometry Penalty ($L_{penalty}$):** Penalises any cell if average water depth (V / Area) exceeds maximum depth (h).
- **4. 2-Step Pushforward Trick ($L_{stability}$):** To stop tiny errors from snowballing over 45 steps, run 2 steps during training: feed Step 1's slightly noisy prediction into Step 2 and train Step 2 to recover!

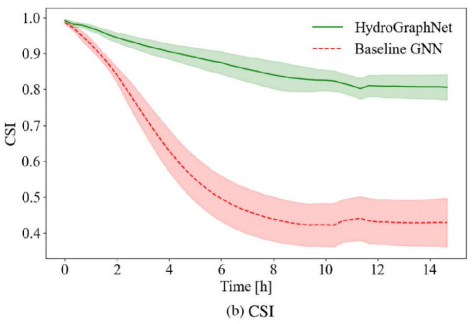
Paper's Headline Results (15-Hour Flood Rollout)

- **650× Faster Than Physics Solver:** Predicts a full 15-hour flood in 0.20 seconds vs. ~130 seconds in HEC-RAS.
- **3× Lower Error Than Baseline GNN:** Reported 0.18 m average RMSE (NSE = 0.89) vs. 0.55 m RMSE for their baseline GNN (curves on bottom-right).

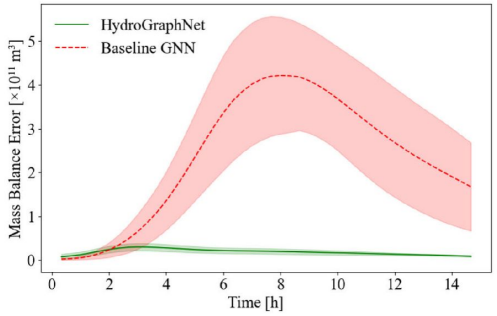




(a) RMSE



(b) CSI



(c) Mass Balance Error

Training Losses & Pushforward Stability

The paper trains with 4 loss terms

1. Prediction loss L_pred

MSE of predicted vs. true [Δh, ΔV] at every node.

2. Physics loss L_physics

Global water conservation — the whole-domain tank must balance.

3. Penalty loss L_penalty

Per-cell consistency — mean depth ≤ max depth.

4. Pushforward stability L_stability

2-step autoregressive stability with perturbed inputs.

1. Node Prediction Loss (Eq. 17 — Depth & Normalized Volume MSE)

$$\mathcal{L}_{\text{pred}} = \frac{1}{|V|}\sum_{v\in V}\left[\left(\hat{h}_v^{(t)} - h_v^{(t)}\right)^2 + \left(\frac{\hat{v}_v^{(t)}}{A_v} - \frac{v_v^{(t)}}{A_v}\right)^2\right]$$

2. Global Continuity Physics Loss (Eqs. 19-24 — Forward & Backward Basin Tank Bounds)

$$s^{(t)} = i_{\text{up}}^{(t)} + r^{(t)} A_{\text{tot}}, \qquad \mathcal{L}_{\text{physics}} = \mathcal{L}_t^{(1)} + \mathcal{L}_t^{(2)}$$

$$\mathcal{L}_t^{(1)} = \left[\frac{1}{A_{\text{tot}}}\,\text{ReLU}\left(\sum_{v\in V}\hat{v}_v^{(t)} - \sum_{v\in V} v_v^{(t-1)} - \Delta t\left(\frac{s^{(t-1)}+s^{(t)}}{2} - \sum_{v\in V}\bar{i}_v A_v\right)\right)\right]^2$$

$$\mathcal{L}_t^{(2)} = \left[\frac{1}{A_{\text{tot}}}\,\text{ReLU}\left(\sum_{v\in V} v_v^{(t+1)} - \sum_{v\in V}\hat{v}_v^{(t)} - \Delta t\left(\frac{s^{(t)}+s^{(t+1)}}{2} - \sum_{v\in V}\bar{i}'_v A_v\right)\right)\right]^2$$

3. Subgrid Cell Consistency Penalty (Eq. 25 — Mean Depth <= Max Depth)

$$\mathcal{L}_{\text{penalty}} = \frac{1}{|V|}\sum_{v\in V}\left[\max\left(0,\ \frac{\hat{v}_v^{(t)}}{A_v} - \hat{h}_v^{(t)}\right)\right]^2$$

4. Two-Step Pushforward Stability & Total Loss (Eqs. 28-29)

$$\mathcal{L}_{\text{stability}} = \frac{1}{|V|}\sum_{v\in V}\left(\mathcal{A}(u^{(t)}+\epsilon) - u^{(t+1)}\right)^2, \quad \mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{physics}} + \mathcal{L}_{\text{penalty}} + \mathcal{L}_{\text{stability}}$$

KAN is just MLP with learnable activations – used for ‘Interpretability’

But it can't multiply features together, and only 1 layer is KAN while the rest are MLPs

WHAT IS KAN?

KAN (Kolmogorov-Arnold Networks) works like MLP, but instead of fixed activation functions (like ReLU), each neuron has its own learnable curve. This means you can 'see' how each input is transformed that's the claimed advantage of 'interpretability'.

WHAT'S THE PROBLEM?

- CAN'T MULTIPLY FEATURES: Flood physics needs combinations like Rain × Area. KAN only adds separate waves per feature — zero cross-feature multiplication!
- ONLY 1 LAYER IS KAN: The code uses just 1 KAN block for the node encoder. The rest (5 Edge MLPs + 5 Node MLPs + 1 Decoder) are standard MLPs — 33 black-box layers!
- MAKES MODEL SLOW & UNSTABLE: More weights, no LayerNorm, and PyTorch JIT disabled

WHY IT'S JUST FOR SHOW

- INTERPRETABILITY IS MISLEADING: The curves you see in Layer 0 can be flipped, rescaled, or cancelled out by the 33 MLP layers after it
- SAME INFORMATION, DIFFERENT SHELL: The downstream MLPs have to learn all interactions anyway — KAN doesn't reduce the learning burden

WHAT SHOULD WE DO INSTEAD

KEEP

- A simple 2-layer MLP + LayerNorm

DROP

- KAN, Fourier basis, cosine oscillation

WHY THE PHYSICS LOSS & PENALTY ARE UNWANTED

It's the same information as the prediction loss — just in another form. Helps in the first few steps (random weights), but becomes useless once the model learns

WHAT DOES L_physics DO?

It uses the continuity equation: Δ(Storage) = Δ(Sources - Sinks). In simple terms: V(t+1) = V(t) + Inflow - Infiltration. The paper introduces two inequalities that check this at the whole-basin level.

THE TWO INEQUALITIES

- FORWARD CONSISTENCY: Predicted volume at time t cannot exceed the previous ground-truth volume plus net incoming water
- BACKWARD CONSISTENCY: Ground-truth volume at t+1 cannot exceed the predicted volume at t plus net incoming water
- WHY IT HELPS INITIALLY: When model parameters are random, these constraints guide the gradients toward physically sensible answers

WHY IT'S UNWANTED

- SAME INFORMATION, DIFFERENT SHELL: L_pred already learns this from the data. The HEC-RAS targets are already locally conservative — the physics loss just re-checks it
- BECOME IDLE: Once the model parameters get better, L_physics becomes zero (no violation) and provides no gradient signal
- The model considers rainfall contribution only over impervious areas using the IP/100 formula. It ignores runoff generated from pervious soil, even when the soil becomes saturated and cannot absorb more water.

2. Global Continuity Physics Loss (Eqs. 19-24 — Forward & Backward Basin Tank Bounds)

$$s^{(t)} = i_{\text{up}}^{(t)} + r^{(t)} A_{\text{tot}}, \qquad \mathcal{L}_{\text{physics}} = \mathcal{L}_t^{(1)} + \mathcal{L}_t^{(2)}$$

$$\mathcal{L}_t^{(1)} = \left[\frac{1}{A_{\text{tot}}} \text{ReLU}\left(\sum_{v \in V} \hat{v}_v^{(t)} - \sum_{v \in V} v_v^{(t-1)} - \Delta t \left(\frac{s^{(t-1)} + s^{(t)}}{2} - \sum_{v \in V} \bar{i}_v A_v\right)\right)\right]^2$$

$$\mathcal{L}_t^{(2)} = \left[\frac{1}{A_{\text{tot}}} \text{ReLU}\left(\sum_{v \in V} v_v^{(t+1)} - \sum_{v \in V} \hat{v}_v^{(t)} - \Delta t \left(\frac{s^{(t)} + s^{(t+1)}}{2} - \sum_{v \in V} \bar{i}'_v A_v\right)\right)\right]^2$$

PENALTY LOSS (L_penalty)

Checks that average depth ≤ max depth: V/A ≤ h. Similar case — it helps initially when parameters are random, but stays idle once the model gets better. Dropping it changes RMSE by only 0.1 mm.

3. Subgrid Cell Consistency Penalty (Eq. 25 — Mean Depth <= Max Depth)

$$\mathcal{L}_{\text{penalty}} = \frac{1}{|V|} \sum_{v \in V} \left[\max\left(0, \frac{\hat{v}_v^{(t)}}{A_v} - \hat{h}_v^{(t)}\right)\right]^2$$

PUSHFORWARD STABILITY

Training the model to handle its own predictions

The Concept

The model predicts future flood levels autoregressively:

$$u(t+1) = A(u(t))$$

During training, each step uses ground truth data.
During inference, each step uses the model's own predictions.

Pushforward stability is a loss term that trains the model to behave consistently in both scenarios:

$$L_stability = ||A(A(u(t-1))) - u(t+1)||^2$$

This encourages the model to predict two steps ahead in a single forward pass, reducing error accumulation.

How It Works

1. Take a real state u(t-1)

2. Apply the model twice:
   $A(u(t-1)) \longrightarrow u(t)$
   $A(A(u(t-1))) \longrightarrow u(t+1)$

3. Compare to actual u(t+1)

4. Minimise the error

Why It Works

The model learns to predict two steps ahead in a single forward pass.

This enforces consistency:

$$A(A(u)) \approx A_direct(u, 2 \text{ steps})$$

During inference, chained predictions stay close to direct ones.

PUSHFORWARD STABILITY

Training the model to handle its own predictions

Mathematical Derivation

Start with the model prediction at time t:

$u(t) + \varepsilon = A(u(t-1))$ (we expect this)

Therefore, the error/perturbation is:

$\varepsilon = A(u(t-1)) - u(t)$

It represents the difference between the model-generated state and the actual state.

Substituting into the stability loss:

$L_stability = ||A(u(t) + \varepsilon) - u(t+1)||^2$

gives:

$L_stability = ||A(A(u(t-1))) - u(t+1)||^2$

The Problem

This requires multiple forward passes and backward passes, making the model slow.

The Solution

Instead, add a small Gaussian perturbation/random noise.

$L_stability = ||A(u(t) + \varepsilon) - u(t+1)||^2$

Where ε is a small gaussian noise.

This increases speed 3x with nearly the same results.

The Bug

There is a serious bug in this part of the code implementation: they used a misaligned target.

Stripping HydroGraphNet Down to What Actually Needed

Keep the physical graph + Δh core; replace KAN, physics penalties & 2-step pushforward with clean pure PyTorch

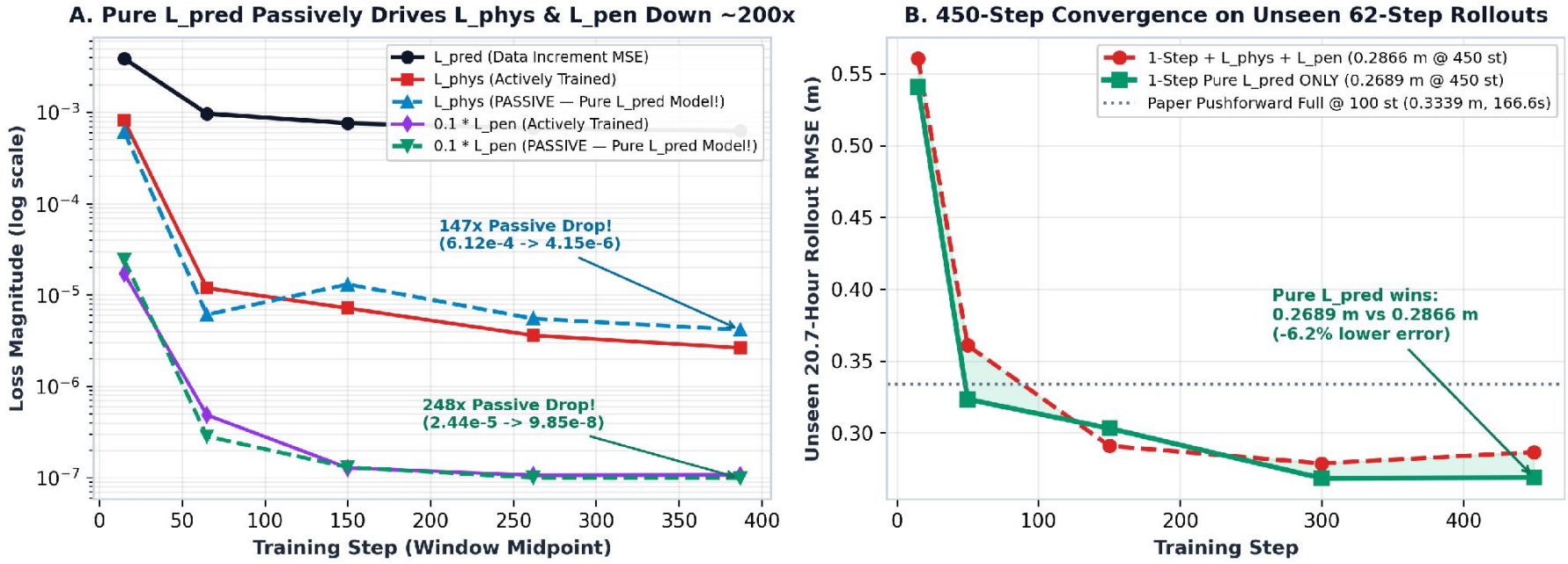
Model Piece	Original Paper / NVIDIA Repo	Clean Rewrite (sidhu2690/HydroGraphNet)
Graph & Inputs	4,787 nodes, k=4 edges, 16 node / 3 edge features	KEPT! (Plus fixed weather alignment to step s+1)
Output Prediction	Predicts 20-min changes [Δh, ΔV] + adds to h_t, V_t	KEPT! (Core reason GNN flood forecasting works)
Node Encoder	**1-layer Fourier KAN (no LayerNorm, no feature mixing)**	**2-Layer MLP + SiLU + LayerNorm (clean & smooth)**
Training Loss	**L_pred + L_physics + L_penalty + L_stability**	**Pure 1-Step Data Loss L_pred ONLY!**
Long-Rollout Stability	**2-Step Pushforward (3 GNN passes per batch)**	**1-Step Random-Walk Noise**
Software Stack	physicsnemo + PyG, etc	100% Pure PyTorch

Proof #1

Identical-seed (seed=42) test on White River M80: what happens to L_phys & L_pen when we NEVER train on them?

Controlled Experiment Setup

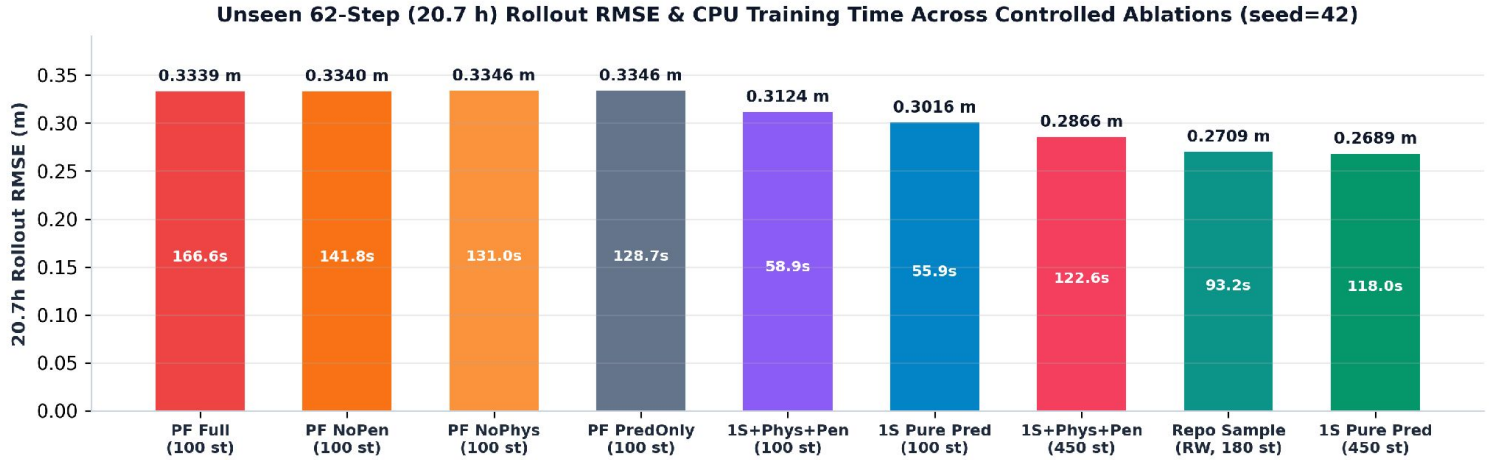
- Both models use the exact same White River M80 mesh (4,787 nodes), same seed=42 weights (214,210 params), and same training storms (H1–H6), tested on full 62-step (20.7-hour) unseen flood rollouts (T1–T3).
- **Key Discovery (Left Plot):** Even when L_phys and L_pen are turned COMPLETELY OFF (weight = 0, dashed curves), watching them passively shows L_phys drops 147× and L_pen drops 248× automatically while achieving 6.2% LOWER flood error (Right Plot: 0.2689 m vs 0.2866 m)!



Proof #2

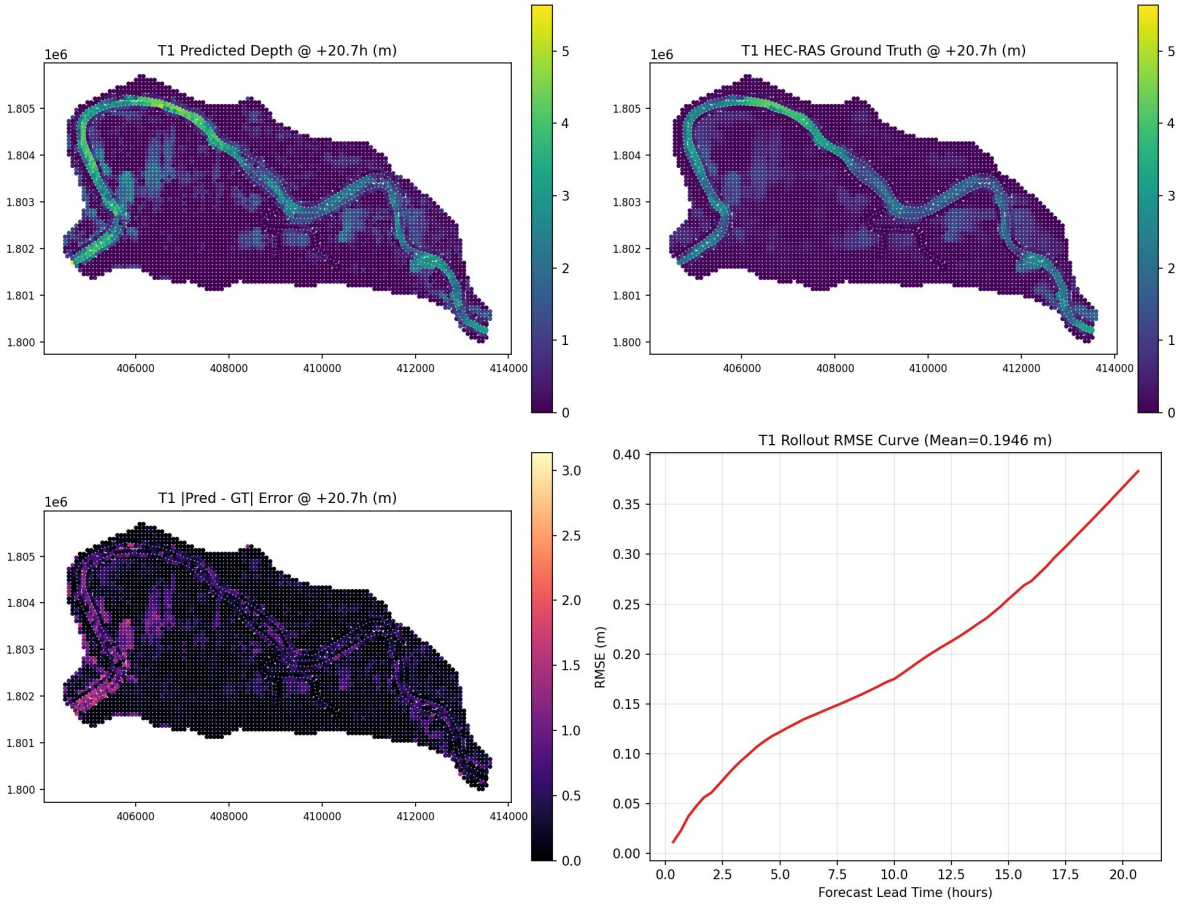
Every controlled variant tested on unseen 20.7-hour (62-step) White River flood rollouts in real physical units (meters)

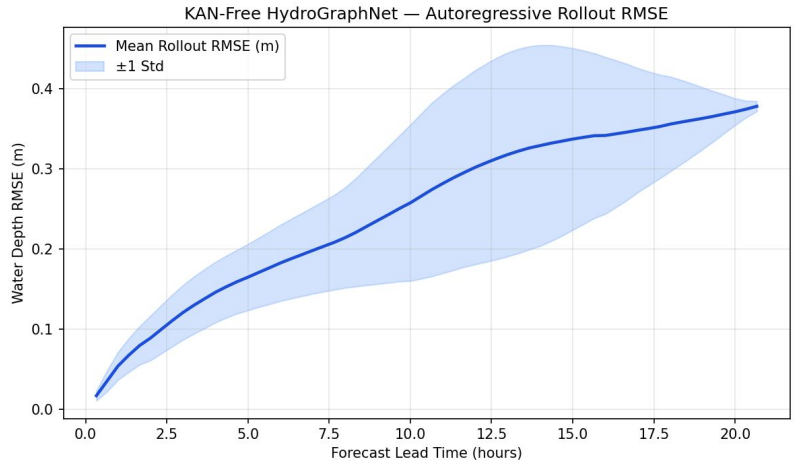
Model / Training Setup	What Is Included?	Time	5h RMSE	20.7h RMSE ↓	NSE ↑	CSI@.30 ↑	Vol Err ↓
1. Paper PF Full (100 st)	**Pushforward + L_pred + L_phys + L_pen**	**166.6 s**	**0.1194 m**	**0.3339 m**	**0.7850**	**0.6284**	**34.35%**
2. PF — No L_pen (100 st)	Pushforward + L_pred + L_phys	141.8 s	0.1195 m	0.3340 m	0.7848	0.6282	34.27%
3. PF — Pure Data (100 st)	Pushforward + L_pred only	128.7 s	0.1193 m	0.3346 m	0.7837	0.6230	32.53%
4. 1-Step + Physics (100 st)	1-Step + L_pred + L_phys + L_pen	58.9 s	0.1197 m	0.3124 m	0.8189	0.6484	18.99%
5. 1-Step Pure Data (100 st)	**1-Step + L_pred ONLY (3.0× faster!)**	**55.9 s**	**0.1185 m**	**0.3016 m**	**0.8311**	**0.6704**	**17.61%**
6. 1-Step Pure Data (180 st)	1-Step + L_pred ONLY (more steps, <PF time)	90.0 s	0.1184 m	0.2812 m	0.8533	0.7532	15.86%
7. Long + Physics (450 st)	1-Step + L_pred + L_phys + L_pen	122.6 s	0.1130 m	0.2866 m	0.8456	—	26.93%
8. Long Pure Data (450 st)	**1-Step + L_pred ONLY (Best Accuracy!)**	**118.0 s**	**0.1106 m**	**0.2689 m**	**0.8651**	**0.7267**	**23.00%**
9. Clean Repo Default (180 st)	**1-Step + Random-Walk Noise + Pure L_pred**	**93.2 s**	**0.1142 m**	**0.2709 m**	**0.8653**	**0.7336**	**13.71%**



Seeing the Simpler KAN-Free Model Predict a 20.7-Hour Flood

Zero-shot rollout on unseen test storms (T1–T3) — trained in 93 seconds on CPU without KAN, L_phys, L_pen, or Pushforward





Unseen 20.7-Hour (62-Step) Forecast Accuracy in Real Meters

Unseen Storm	RMSE (m)	MAE (m)	NSE ↑	CSI@.05	CSI@.30	VolErr
Storm T1	0.2170	0.1363	0.8860	0.7064	0.7433	8.49%
Storm T2 (Huge)	0.3694	0.2187	0.8184	0.7663	0.7187	24.09%
Storm T3	0.2262	0.1436	0.8915	0.7241	0.7387	8.57%
Average (T1–T3)	**0.2709**	**0.1662**	**0.8653**	**0.7323**	**0.7336**	**13.71%**

What You Are Looking At (Left)

- **Spot-On Flood Map After 62 Self-Fed Steps:** Predicted river & floodplain depths at +20.7 hours match HEC-RAS ground truth closely (~22 cm error on T1 & T3, <8.6% volume error)!